FPGA-based Intrusion Detection for DoS Attack

Eoin Sheerin
ID: 20465956
eoin.sheerin.2021@mumail.ie

Department of Electronic Engineering
National University of Ireland, Maynooth
Ireland

Submitted to the Department of Electronic Engineering in partial fulfilment of the requirements for the Master of Electronic Engineering Meng(Hons) degree

Facilitator: Dr. Erivelton Nepomuceno

Word Count: (x)

Declaration

I have read and understood the Departmental policy on plagiarism.

I declare that this thesis is my own work and has not been submitted in any form for another degree or diploma at any university or other institution of tertiary education.

Information derived from the published or unpublished work of others has been acknowledged in the text and a list of references is given.

Student Signature:

Eoin Sheerin

Student Number:

20465956

Date:

05/2026

Acknowledgements

First and foremost, I would like to express my sincere gratitude to my supervisor, Dr. Erivelton Nepomuceno, for allowing me the opportunity to do this project, as well as your willingness to assist me in this matter.

I am also deeply grateful to Dr. John Dooley for providing the necessary hardware resources required for this research. His technical assistance and advice in overcoming implementation challenges were greatly appreciated.

Furthermore, I would like to thank the faculty and staff within the Department of Electronic Engineering at Maynooth University for their continued support and for providing a positive environment during my studies.

Finally, I would like to thank my family for their support and encouragement.

Abstract

This report presents the design and implementation of an FPGA-based Intrusion Detection System, which has the ability for real-time detection of Denial of Service attacks. This report outlines the core design challenges encountered during development of this project and also details the solutions implemented to address them. Testing was also conducted to evaluate the system's performance, with the resulting data proving its long-term reliability and operational stability.

Examples of existing intrusion detection approaches are discussed, along with relevant technical background, to provide context for the design choices made in this project. Particular focus is given to the advantages and limitations of implementing an IDS on FPGA hardware compared to traditional software-based solutions.

This project demonstrates the potential of hardware-accelerated intrusion detection, highlighting the benefits of low-latency processing and parallel execution for network security applications.

Contents

Declaration **i**

Acknowledgements **ii**

Abstract **iii**

Contents **iv**

List of Figures **v**

1 Introduction **1**
1.1 Problem Statement and Project Solution . . . 2
1.2 Report Structure . . . 3

2 Technical Background **4**

3 Literature Review **5**

4 System Design and Development **6**
4.1 High Level System Overview . . . 6
4.2 Packet Replay from BRAM . . . 7
4.3 Packet Parsing . . . 10
4.3.1 Packet structure and assumptions . . . 10
4.3.2 Streaming interface behaviour . . . 10
4.3.3 Word-index based extraction . . . 10
4.3.4 Protocol classification logic . . . 10
4.3.5 Verilog implementation . . . 11
4.3.6 Design considerations and challenges . . . 11
4.3.7 Output of the parser . . . 11
4.4 Feature Extraction and Traffic Monitoring . . . 11
4.5 Sliding Window Detection . . . 11
4.6 Volumetric Based Detection Logic . . . 11
4.7 Microblaze integration . . . 11
4.8 GUI . . . 11

5 Results and Testing **12**
5.1 Testing . . . 12
5.1.1 Packet Representation for a FPGA system . . . 12
5.1.2 Packet Dataset and Test Setup . . . 13
5.2 Results . . . 14
5.2.1 Threshold and Packet Rate Analysis . . . 14
5.2.2 High Packet Rate: `usleep(5000)` . . . 15
5.2.3 Baseline Packet Rate: `usleep(10000)` . . . 15
5.2.4 Low Packet Rate: `usleep(20000)` . . . 15
5.2.5 Discussion . . . 16
5.2.6 Syn Flood . . . 16
5.2.7 Mixed Packets . . . 17
5.2.8 ICMP Flood . . . 19
5.2.9 NULL Scan Detection . . . 19
5.2.10 XMAS Scan Detection . . . 20
5.2.11 UDP Flood Detection . . . 21
5.3 System Limitations . . . 21
5.4 Comparison with Software-Based IDS . . . 22

6 Ethical Considerations **23**

7 Conclusions and Future Work **24**

List of Figures

1 DDoS attack diagram [10] . . . 1
2 Spike in traffic diagram [12] . . . 2
3 Block diagram of the FPGA-based intrusion detection system architecture . . . 6
4 Annotated Vivado block diagram of the FPGA-based IDS system . . . 9
5 Comparison of packet representations. The original PCAP format contains full protocol information, while the compact format stores packets as fixed 32-bit words for FPGA replay. . . . 12
6 A Typical IPv4 TCP header . . . 13
7 Example packets used in testing. Left: normal TCP traffic. Right: TCP SYN packet, showing the SYN flag used for attack detection. . . . 14
8 Extracted destination ports from the SYN packet dataset using `tcpdump`. The repeated destination IP with varying ports indicates SYN-based scanning or probing behaviour. . . . 16
9 Mixed traffic scenario generated using the IDS attack scenario builder . . . 17
10 Window-level output for mixed traffic scenario . . . 18

1 Introduction

Information and Communication Technology (ICT) services are a fundamental part of modern society, supporting critical infrastructure across sectors such as healthcare, government, and finance [1]. As noted by the World Economic Forum, this increasing reliance has expanded the overall potential digital attack surface, making systems more susceptible to malicious disruption [2].

While common cyber threats such as malware, phishing, and ransomware primarily target data integrity and confidentiality, attacks that target system availability remain a major concern. According to the ENISA Threat Landscape 2024 report, Denial of Service (DoS) and Distributed Denial of Service (DDoS) attacks "play a significant role in the cybersecurity threat landscape" [3]. These attacks aim to overwhelm network resources, preventing legitimate users from accessing essential services.

DoS (Denial of Service) attacks are a class of cyberattack in which an attacker attempts to render a system or network resource unavailable to legitimate users by overwhelming it with malicious traffic [4]. A Distributed Denial of Service (DDoS) attack extends this concept by originating from multiple sources simultaneously, significantly increasing the scale and impact of the attack [5]. These attacks are among the most disruptive threats to network availability, capable of causing large-scale service outages and significant financial and reputational damage [6].

In 2020, experts identified 1,560,000 more DDoS attacks than in 2019 as described by [6]. As shown in Figure 1, an attacker can use a network of compromised devices, known as a botnet. Botnets are small malicious programs that are created and controlled by an attacker [7] A botnet consists of multiple infected hosts, each running malicious software that can be remotely controlled by an attacker [7]. This distributed nature provides scalability and anonymity, making DDoS attacks particularly difficult to mitigate using traditional defence mechanisms [8]. DoS attacks exploit vulnerabilities in network infrastructure and can be launched using various methods. The main goal of a DoS attack is to disrupt normal operations, which can cause system downtime which results in service interruptions, revenue loss, and damage to the organization's reputation and cause downtime and potentially inflict financial or reputational damage [9].

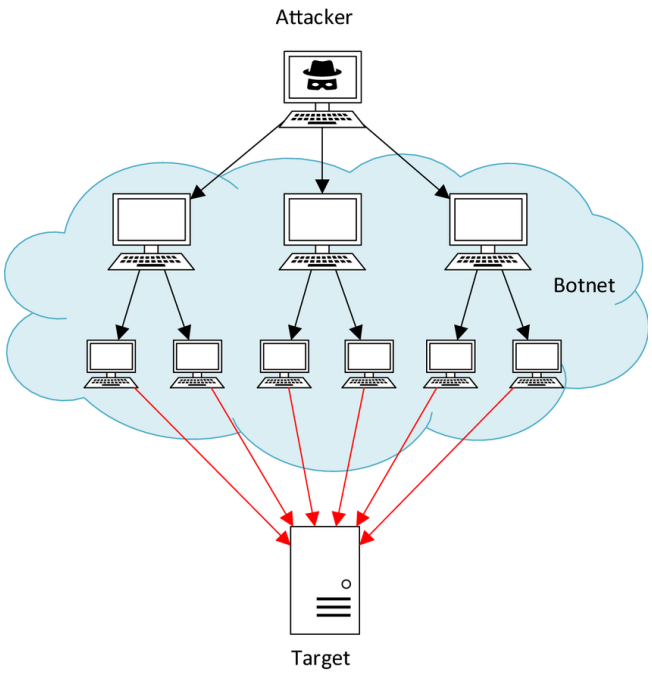


Figure 1: DDoS attack diagram [10].

An effective approach to network security requires consideration of multiple layers within an organisation's infrastructure, as attacks may target different points within the network stack (the hierarchical implementation of networking protocols (e.g., TCP/IP) in software or firmware, managing data transmission from an application down to the physical network). Network attacks are unauthorised actions aimed at compromising the confidentiality, integrity, or availability of data and services. These attacks can broadly be classified as either passive or active. Passive attacks involve the interception of data without modification, whereas active attacks involve altering, disrupting, or destroying network traffic or system resources [11].

DDoS attacks fall into the category of active attacks, as their primary objective is to disrupt normal system operation by overwhelming network resources. Mitigation strategies for such attacks can be implemented using either software-based or hardware-based approaches, each offering different trade-offs in terms of performance, scalability, and cost. Software-based solutions provide flexibility and ease of deployment, while hardware-based approaches offer higher throughput and lower latency, making them better suited to handling high-speed network traffic.

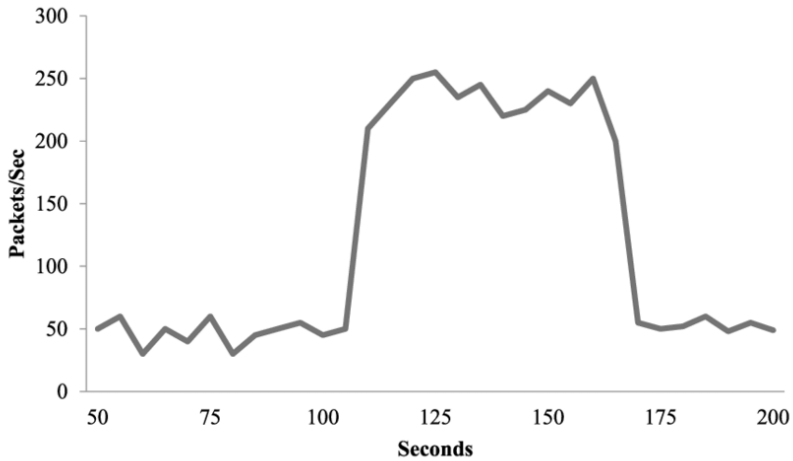


Figure 2: Spike in traffic diagram [12].

Detecting these attacks is a complex task, as it requires monitoring high volumes of traffic, distinguishing between legitimate and malicious requests, and responding to rapidly evolving attack methods. One of the main challenges is the scale of traffic generated during an attack. As shown in Fig. 2, attacks are characterised by sudden and significant spikes in packet rates compared to normal network behaviour. Such surges can overwhelm both network infrastructure and processing systems, making real time analysis difficult.

In addition to high traffic volume, distributed botnets obscure the true origin of attacks and often employ evasion techniques that mimic legitimate traffic patterns, further complicating detection [13]. To address these issues a range of detection/mitigation techniques have been developed. These include anomaly detection, which identifies unusual traffic behaviors that deviate from normal patterns [14], rate limiting, a technique that restricts the number of requests from a single source within a given time frame to prevent overloading a server [15], and traffic filtering, which blocks malicious packets based on predefined rules or signatures [16].

However, implementing these techniques at scale requires significant computational resources. Large-scale DDoS attacks can generate traffic volumes in the terabit per second range, requiring inspection of millions of packets per second (pps). For example, recent reports show that Cloudflare mitigated a 7.3 Tbps DDoS attack, delivering approximately 37.4 terabytes of data in just 45 seconds [17]. Processing traffic at this scale places substantial demands on detection systems, as packets must be analysed in real time. This shows the need for high-performance detection mechanisms capable of operating at line rate, motivating the use of hardware-accelerated solutions such as FPGAs.

Traditional software-based defenses running on general-purpose Central Processing Units (CPUs) face inherent limitations. As described by [18], CPU based systems, even those with multi-core architectures, face challenges in scaling because of the "Von Neumann bottleneck" which is where the CPU must sequentially interrupt its processing to fetch packet data from memory [19]. This can cause the CPU to enter into a state of "receive livelock" where the processor consumes all available interrupts, leaving no resources for other necessary tasks [20]. This can cause the defence system to crash, in fact industry reports show that over 80% of interviewed respondents firewalls have crashed during DDoS attacks [21].

1.1 Problem Statement and Project Solution

The goal of this project was to develop a Field Programmable Gate Array (FPGA) based Intrusion Detection System (IDS) on the Xilinx VC707 platform, capable of detecting Denial of Service (DoS) attacks in real-time. The system implements a packet inspection pipeline designed to overcome the limitations of traditional software-based approaches.

Modern software-based defenses are limited in their ability to process high volumes of network traffic due to

CPU bottlenecks, interrupt overhead, and the sequential nature of packet processing. This motivates the use of FPGA-based architectures, which enable parallel processing and low-latency analysis, making them well-suited for high-throughput network monitoring.

A "monitor-analyse-mitigate" framework is implemented, where incoming packet streams are monitored in real-time, analysed for anomalous behaviour, and used to generate alerts indicating potential DoS attacks. Packet data is replayed from on-chip Block RAM (BRAM) and processed by a hardware packet parser, which extracts key features including protocol type, destination IP address, and TCP flags. These features are then evaluated using a sliding window detection mechanism, where packet counts and protocol-specific activity are aggregated over fixed time intervals.

Detection is performed using configurable thresholds for both volumetric and protocol-based attacks. The system is capable of identifying multiple attack types, including SYN floods, ICMP floods, UDP floods, and TCP-based scan attacks such as NULL and XMAS scans. Upon detection, alert signals are generated in hardware and communicated to an on-chip MicroBlaze processor, which aggregates results and outputs them to a host system via a UART interface.

Testing was conducted using both synthetic and real-world network traffic datasets, including CICIDS2017 and CICIDS2019 captures, to evaluate system behaviour under a range of attack scenarios. The system was assessed in terms of packet classification accuracy, sliding window detection performance, and responsiveness to varying traffic rates.

1.2 Report Structure

The rest of this report is organised as follows:

Section 2 Covers the technical background necessary to understand the proposed solution.

Section 3 Provides a literature review of relevant literature, as well as discussing previous projects trying to achieve similar objectives.

Section 4 Presents the complete system design and development.

Section 5 Presents the testing performed on the system described in Section 4, along with a discussion of the results.

Section 6 Discusses the ethical considerations associated with the project and the developed system.

Section 7 Provoides the conclusion to the report by presenting an evaluation of the project and outlining potential areas for future work.

2 Technical Background

3 Literature Review

4 System Design and Development

This section presents the design and development of the FPGA-based IDS. It describes the architectural decisions, hardware implementation, and software integration required to achieve real-time detection of DoS attacks. The system is designed as a packet processing pipeline, incorporating packet parsing, feature extraction, and a sliding window-based detection mechanism. Each component of the system is detailed, along with the design considerations and challenges encountered during development.

4.1 High Level System Overview

The FPGA-based intrusion detection system is designed as a data processing pipeline for real-time analysis of network traffic. Rather than relying on sequential software processing, the system performs packet inspection directly in hardware, enabling high-throughput and low-latency detection of DoS attacks.

Network traffic is supplied to the system by the MicroBlaze processor, which replays pre-processed packet datasets stored as C header files into the FPGA packet input interface. As packets are streamed into the system, a hardware packet parser extracts key fields such as protocol type, destination IP address, and TCP flags. These features are then passed to a detection module, which implements a sliding window-based approach to analyse traffic patterns over fixed time intervals.

Within each detection window, packet counts and protocol-specific activity are aggregated and compared against thresholds to identify malicious behaviour. When an attack condition is met, the system flags the detected behaviour in hardware and outputs status signals with the detected attack type, which are then read by the MicroBlaze processor for aggregation and reporting.

The output from the hardware detection logic is communicated to a MicroBlaze processor, which performs higher-level aggregation and reporting. Detected attacks and traffic statistics are then transmitted to the serial monitor via a UART interface.

A block diagram of the overall system architecture is shown in Fig. 3.

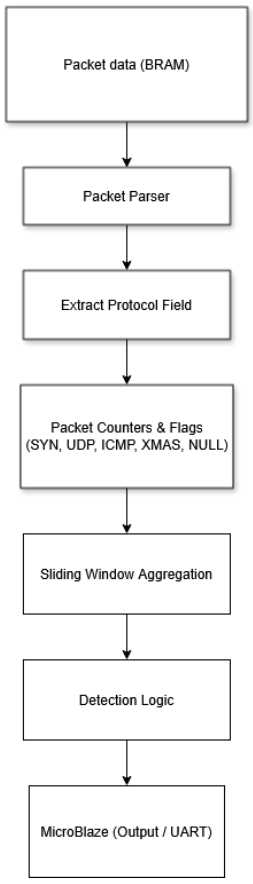


Figure 3: Block diagram of the FPGA-based intrusion detection system architecture

The system architecture shown in Fig. 3 shows the flow of packet data through the FPGA-based IDS. Packet

data is first retrieved by the MicroBlaze processor from the selected software defined dataset and passed through a hardware packet parser, where protocol fields are extracted. These extracted features are then used to update protocol-specific counters, which are aggregated over a sliding time window to analyse traffic behaviour.

The detection logic looks at the aggregated statistics against predefined thresholds to identify potential DoS attacks. When an attack condition is met, the system generates detection flags, which are read by the MicroBlaze processor for further processing and communication via UART. This pipelined architecture enables efficient, real-time analysis of network traffic.

4.2 Packet Replay from BRAM

The IDS uses on-chip Block RAM (BRAM) as a source of network traffic for testing. Packet data is pre-defined and stored in memory, then replayed into the FPGA to emulate real network traffic conditions.

Each packet is stored in a C header file as a sequence of 32-bit words. The active dataset is selected in the Vitis application, allowing different packet sets to be tested by changing the included header file and associated dataset definition. During execution, the MicroBlaze processor reads the selected packet records from software memory and writes each 32-bit word to the FPGA packet input interface.

Listing 1 shows the packet representation used in the software-defined datasets. Each packet contains a length field and an array of 32-bit words, allowing packets of different sizes to be replayed through the same transmission function.

Control signals, including `TVALID` and `TLAST`, are asserted during this transfer to indicate valid data and the final word of a packet, respectively. This enables the hardware modules to correctly interpret packet boundaries and process incoming data in a structured manner.

In real-world network interfaces, such as Ethernet Media Access Control (MAC) layers, packet boundaries are defined by frame structures within the hardware layer [22]. The use of control signals in this system therefore emulates real-world packet streaming behaviour within a controlled test environment.

Packet transmission is managed by the MicroBlaze processor, which reads packet data from memory and writes it to the hardware interface via memory-mapped GPIO. This allows precise control over packet timing and ordering, enabling different traffic scenarios, including both normal and malicious patterns, to be evaluated.

Initial approaches for sending packet data to the system involved the use of pre-initialised memory formats (e.g. `.coe` and `.hex`). This was implemented using a Vivado Distributed Memory Generator configured as a ROM, with packet data preloaded via memory initialization files. While this approach allowed static dataset initialisation within the hardware design, it required regeneration of bitstream and re-synthesis when switching between datasets.

Additionally, the ROM introduced a one-cycle read delay, meaning that data was not available until the following clock cycle. This caused the parser to read the wrong word positions during early development, leading to incorrect extraction of protocol fields, for example TCP flags.

To address this, the design was changed to use a runtime packet replay mechanism. Packet data is defined in software and sent to the FPGA by the MicroBlaze processor, rather than being preloaded into memory. This makes it easier to switch between datasets during testing without requiring hardware reconfiguration, while also ensuring that packet data is correctly aligned during processing.

The selected packet record is transmitted to the FPGA one word at a time, as shown in Listing 2. The final word of each packet is marked using `TLAST`.

Listing 1: Example packet representation used in the C header datasets

```
static const packet_record_t syn_packet_real_world[] = {
  {
    .len_words = 15,
    .words = {
      0x90B11CA2,
      0xC0D370F3,
      ...
    }
  }
};
```

Listing 2: Packet transmission using TVALID and TLAST control signals

```
static void send_packet(const packet_record_t *pkt) {
  for (int i = 0; i < pkt->len_words; i++) {
    uint32_t ctrl = CTRL_TVALID;
    if (i == pkt->len_words - 1)
      ctrl |= CTRL_TLAST;

    Xil_Out32(GPIO_PKT_BASE, pkt->words[i]);
    Xil_Out32(GPIO_CTRL_BASE, ctrl);
    usleep(10);
    Xil_Out32(GPIO_CTRL_BASE, 0);
  }
}
```

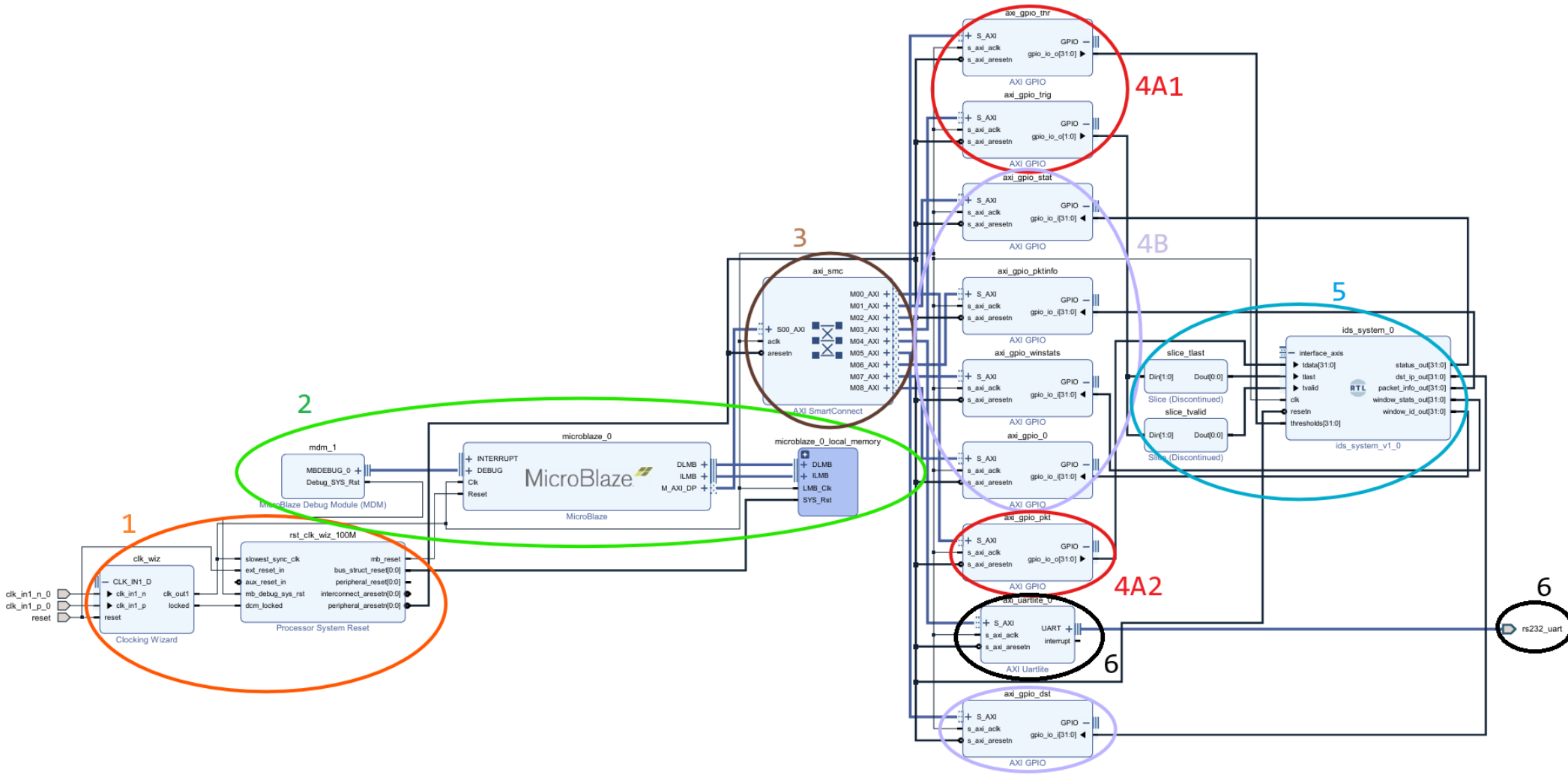


Figure 4: Annotated Vivado block diagram of the FPGA-based IDS system

The annotated Vivado block diagram shown in Fig. 4 illustrates the overall architecture of the system. For clarity, key subsystems are highlighted using coloured regions and labelled accordingly.

(1) Clock and Reset Generation (orange): Provides a stable clock signal and synchronised reset for all components, ensuring reliable system operation.

(2) MicroBlaze Processor System (green): Acts as a software-based MicroBlaze processor, serving as the central control unit for packet replay, configuration of detection thresholds, and collection of detection results.

(3) AXI Interconnect (brown): Allows communication between the MicroBlaze processor and all peripheral modules through AXI interfaces.

(4A) AXI GPIO Inputs (red): Used to transmit packet data, control signals, and configuration parameters from the processor to the hardware modules. In particular, **(4A1)** corresponds to control and configuration interfaces (e.g. thresholds and trigger signals), while **(4A2)** is used for packet data input, where individual packet words are streamed from the MicroBlaze processor into the IDS hardware.

(4B) AXI GPIO Outputs (purple): Return detection results and traffic statistics from the hardware modules back to the processor.

(5) Custom IDS Hardware Module (blue): Implements packet parsing, feature extraction, and attack detection in hardware, processing incoming data streams in real time.

(6) UART Output Interface (black): Sends detection results to an external system via UART.

4.3 Packet Parsing

The packet parsing module is responsible for extracting relevant protocol fields from incoming packet data streams. This forms a critical stage in the system, as it converts raw packet data into structured information that can be used for detection of malicious traffic patterns.

4.3.1 Packet structure and assumptions

The parser operates under the assumption that packets follow a standard Ethernet frame structure with IPv4 encapsulation. As a result, key protocol fields are located at fixed positions within the packet header. This allows the design to avoid complex parsing logic and instead rely on fixed offsets within the data stream.

4.3.2 Streaming interface behaviour

Packets are received as a sequence of 32-bit words via a streaming interface, with `TVALID` indicating valid data and `TLAST` marking the final word of a packet. This enables the parser to process packets sequentially without requiring full buffering of packet contents.

4.3.3 Word-index based extraction

A word index counter is used to track the position within each packet as it is received. Based on this index, specific protocol fields are extracted at known offsets within the packet structure. This approach allows for efficient hardware implementation, as field extraction is performed using simple combinational logic rather than complex parsing algorithms.

For example, the EtherType field is extracted from word 3, the IP protocol from word 5, and TCP flags from word 11. By using fixed offsets, the parser can quickly identify packet types and relevant header information with minimal latency.

4.3.4 Protocol classification logic

Using the extracted fields, the parser classifies packets based on protocol type. IPv4 packets are identified using the EtherType field, after which the IP protocol field is used to distinguish between TCP, UDP, and ICMP traffic.

For TCP packets, additional inspection of the TCP flags field allows the system to identify specific patterns such as SYN packets, as well as scan-based behaviour including NULL and XMAS packets. These classifications form the basis for subsequent detection logic.

4.3.5 Verilog implementation

Listing 3: Extraction of protocol fields using word indexing

```
case (word_index)
  8'd3:  ethertype <= tdata[31:16];
  8'd5:  ip_proto  <= tdata[7:0];
  8'd7:  dst_ip[31:16] <= tdata[15:0];
  8'd8:  dst_ip[15:0]  <= tdata[31:16];
  8'd11: tcp_flags <= tdata[7:0];
endcase
```

As shown in Listing 3, protocol fields are extracted based on the current value of the `word_index` signal. This ensures that each field is captured at the correct position within the packet stream.

4.3.6 Design considerations and challenges

A key design consideration was ensuring correct alignment between the word index and incoming data. During early development, the use of ROM-based packet storage introduced a one-cycle read delay, which caused incorrect field extraction due to misalignment between expected and actual data positions.

This issue was resolved by transitioning to a streaming-based packet replay mechanism, ensuring that packet data arrives in a correctly aligned manner. This allowed the parser to operate reliably using fixed word offsets.

4.3.7 Output of the parser

The output of the parsing module consists of protocol indicators and packet classification signals, including identification of TCP, UDP, and ICMP traffic, as well as specific TCP flag patterns. These signals are passed to the next stage of the system, where they are used for feature extraction and detection of potential attacks.

4.4 Feature Extraction and Traffic Monitoring

4.5 Sliding Window Detection

4.6 Volumetric Based Detection Logic

4.7 Microblaze integration

4.8 GUI

5 Results and Testing

A designed system must be tested to determine its effectiveness in solving the intended problem. This section presents the testing methodology used to assess the FPGA-based intrusion detection system described in Section 4, followed by an analysis of its performance under a range of traffic conditions and attack scenarios.

5.1 Testing

The FPGA-based intrusion detection system was tested to determine its effectiveness in identifying malicious network activity using both controlled and real-world conditions. The testing mainly focuses on two main aspects, per-packet classification accuracy and sliding window based attack detection.

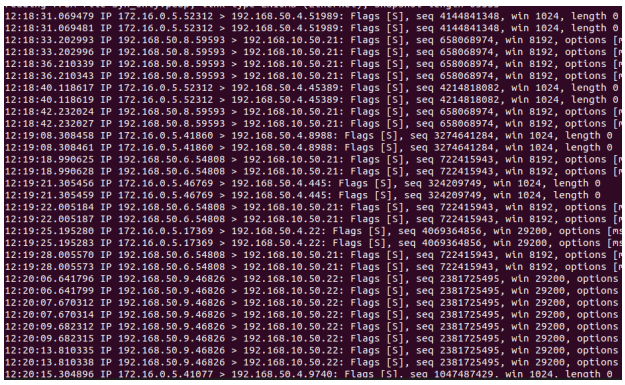
Datasets were constructed from real world PCAP (packet capture) files and then converted into a hex-based format suitable for FPGA-based testing. The packets in their original states are usable with a slight tweak to the verilog code however a hex based format was due to memnory constraints in the FPGA. Each dataset represents a specific attack type, allowing controlled evaluation of detection performance. The following sections present the methodology and results for each traffic type individually, followed by an analysis of overall system behaviour.

5.1.1 Packet Representation for a FPGA system

The initial system design intended to use the onboard Ethernet (RJ45) interface to enable real time packet capture and processing. However, implementation of this approach required access to a licensed Ethernet axi IP core, which was not available within the constraints of this project.

As a result, an alternative approach was used in which packet traces were pre recorded and replayed from on-chip memory (BRAM). This approach provided a repeatable method for evaluating the IDS, allowing testing across multiple traffic scenarios taken from real-world datasets and synthetic dataets generated for testing purposes.

While the original packets remained usable in their PCAP-derived form and could, have been processed directly by the FPGA with minor modifications to the parsing logic. However, doing so would require more exhaustive byte level parsing and additional buffering within the hardware. This would not have been feasible given the fact that the VC707 platform provides approximately 37,080 Kb Mb ($37080Kb \div 8 \approx 4.6$MB) of on chip BRAM, as seen on page 7 of the 7 Series FPGAs Data Sheet: Overview [23]. While this is enough for basic buffering and control logic, it is not sufficient to store large packet traces directly, this memory must also support buffering and control logic, the available capacity for packet storage is further reduced in practice. which is why a reduced packet format and smaller datasets were used during testing. Due to the memory limitations of the FPGA, the original PCAP packet format (shown in Fig. 5(a)) was converted into a more memory efficient representation, which is shown in Figure 5(b) below.



(a) Original PCAP packet view

```
static const packet_record_t
syn_packet_real_world[] = {
{
    .len_words = 15,
    .words = {
        0x90B11CA2,
        0xC0D370F3,
        ...
    }
}
};
```

(b) Compact FPGA packet format

Figure 5: Comparison of packet representations. The original PCAP format contains full protocol information, while the compact format stores packets as fixed 32-bit words for FPGA replay.

In standard IPv4 based communication, packets follow a structured format consisting of multiple fields distributed across several bytes. As shown in Fig. 6, the IPv4 header has a minimum size of 20 bytes, containing fields such as source and destination IP addresses and protocol identifiers.

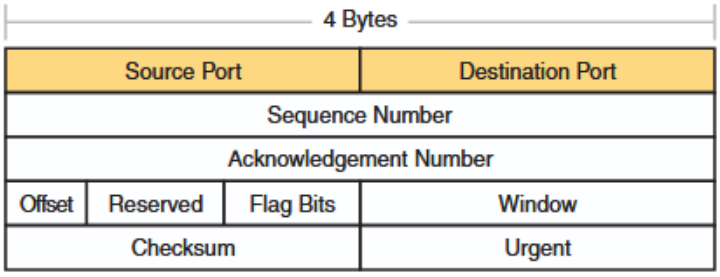


Figure 6: A Typical IPv4 TCP header

While this structure is well suited for general-purpose networking, direct processing of full PCAP byte streams on the FPGA was not adopted in this implementation due to the aforementioned memory constraints. As mentioned previously, the original system design intended to use the onboard Ethernet interface for real-time packet capture and parsing, allowing packets to be processed as they arrive rather than stored in BRAM, and therefore avoiding the associated memory constraints.

To enable better/more testing, packets were transformed into a hex representation in which the fields required for detection were kept and turned into fixed 32-bit words. Key parts of the packet such as EtherType, IP protocol, destination IP address, and TCP flags were encoded directly into predefined positions within each word. In a conventional network, these fields would be extracted by the Ethernet MAC and higher level protocol parsing logic. In this implementation the extraction step is considered already performed, allowing the FPGA to focus on the necesary fields.

5.1.2 Packet Dataset and Test Setup

To evaluate the performance of the FPGA-based intrusion detection system, network traffic datasets were obtained from publicly available intrusion detection benchmarks, specifically the CICIDS2017 and CICIDS2019 datasets. These datasets provide realistic network traffic containing both innocuous activity and also a number of attack behaviours.

Different subsets of the datasets were used to isolate specific traffic types for controlled testing. ICMP and UDP traffic were extracted from the CICIDS2019 Friday Working Hours capture, which contains a mixturse of normal background traffic and network activity representative of real-world conditions. TCP SYN packets and normal traffic were extracted from the CICIDS2017 dataset, allowing evaluation of connection based attacks such as SYN flooding then packets from both datasets were combined to create a file with multiple attack patterns.

The packet captures were processed using `tshark` and `tcpdump` to filter relevant traffic types. Custom Python scripts were then used to convert the filtered packet data into a hardware-compatible format. Each packet was represented as a sequence of 32-bit words and stored in header files, ensuring the playing of the packets through the FPGA.

For consistency across experiments, each dataset was reduced to a fixed size of 64 packets. This ensures repeatability and allows direct comparison between different traffic types and attack scenarios.

Example packets from the dataset are shown in Fig. 7. These illustrate the structure of the packet data used for classification. In particular, the TCP SYN packet (right) is characterised by the SYN flag (0x02), which is used by the IDS to identify connection initiation behaviour and detect potential SYN-based attacks.

```
    Type: IPv4 (0x0800)
Internet Protocol Version 4, Src: 192.168.10.50, Dst: 192.168.10.3
    0100 .... = Version: 4
    .... 0101 = Header Length: 20 bytes (5)
    Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
        0000 00.. = Differentiated Services Codepoint: Default (0)
        .... ..00 = Explicit Congestion Notification: Not ECN-Capable Transport (0)
    Total Length: 455
    Identification: 0xc500 (50432)
    Flags: 0x40, Don't fragment
        0... .... = Reserved bit: Not set
        .1.. .... = Don't fragment: Set
        ..0. .... = More fragments: Not set
    ...0 0000 0000 0000 = Fragment Offset: 0
    Time to Live: 64
    Protocol: TCP (6)
    Header Checksum: 0xdeaa [validation disabled]
    [Header checksum status: Unverified]
    Source Address: 192.168.10.50
    Destination Address: 192.168.10.3
Transmission Control Protocol, Src Port: 56108, Dst Port: 3268, Seq: 1, Ack: 1, Len: 403
    Source Port: 56108
    Destination Port: 3268
    [Stream index: 0]
    [Conversation completeness: Incomplete (0)]
    [TCP Segment Len: 403]
    Sequence Number: 1    (relative sequence number)
    Sequence Number (raw): 1785789788
    [Next Sequence Number: 404    (relative sequence number)]
    Acknowledgment Number: 1    (relative ack number)
    Acknowledgment number (raw): 909172649
    1000 .... = Header Length: 32 bytes (8)
    Flags: 0x018 (PSH, ACK)
        000. .... .... = Reserved: Not set
        ...0 .... .... = Nonce: Not set
        .... 0... .... = Congestion Window Reduced (CWR): Not set
        .... .0.. .... = ECN-Echo: Not set
        .... ..0. .... = Urgent: Not set
        .... ...1 .... = Acknowledgment: Set
        .... .... 1... = Push: Set
        .... .... .0.. = Reset: Not set
        .... .... ..0. = Syn: Not set
        .... .... ...0 = Fin: Not set
        [TCP Flags: ·····AP···]
    Window: 377
    [Calculated window size: 377]
```

```
    Type: IPv4 (0x0800)
Internet Protocol Version 4, Src: 192.168.10.9, Dst: 192.168.10.3
    0100 .... = Version: 4
    .... 0101 = Header Length: 20 bytes (5)
    Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
        0000 00.. = Differentiated Services Codepoint: Default (0)
        .... ..00 = Explicit Congestion Notification: Not ECN-Capable Transport (0)
    Total Length: 52
    Identification: 0x000e (14)
    Flags: 0x40, Don't fragment
        0... .... = Reserved bit: Not set
        .1.. .... = Don't fragment: Set
        ..0. .... = More fragments: Not set
    ...0 0000 0000 0000 = Fragment Offset: 0
    Time to Live: 128
    Protocol: TCP (6)
    Header Checksum: 0x6559 [validation disabled]
    [Header checksum status: Unverified]
    Source Address: 192.168.10.9
    Destination Address: 192.168.10.3
Transmission Control Protocol, Src Port: 1028, Dst Port: 135, Seq: 0, Len: 0
    Source Port: 1028
    Destination Port: 135
    [Stream index: 0]
    [Conversation completeness: Incomplete (0)]
    [TCP Segment Len: 0]
    Sequence Number: 0    (relative sequence number)
    Sequence Number (raw): 3083264464
    [Next Sequence Number: 1    (relative sequence number)]
    Acknowledgment Number: 0
    Acknowledgment number (raw): 0
    1000 .... = Header Length: 32 bytes (8)
    Flags: 0x002 (SYN)
        000. .... .... = Reserved: Not set
        ...0 .... .... = Nonce: Not set
        .... 0... .... = Congestion Window Reduced (CWR): Not set
        .... .0.. .... = ECN-Echo: Not set
        .... ..0. .... = Urgent: Not set
        .... ...0 .... = Acknowledgment: Not set
        .... .... 0... = Push: Not set
        .... .... .0.. = Reset: Not set
        .... .... ..1. = Syn: Set
            [Expert Info (Chat/Sequence): Connection establish request (SYN): server port 135]
                [Connection establish request (SYN): server port 135]
                [Severity level: Chat]
                [Group: Sequence]
        .... .... ...0 = Fin: Not set
```

Figure 7: Example packets used in testing. Left: normal TCP traffic. Right: TCP SYN packet, showing the SYN flag used for attack detection.

Each packet in the dataset is associated with a known classification, including protocol type (TCP, UDP, ICMP) and, where applicable, specific flag combinations (e.g. SYN, NULL, XMAS). This enables direct comparison between expected classifications and the outputs produced by the IDS.

In addition to real-world datasets, synthetic attack traffic was generated using the Nmap (Network Mapper) tool to provide controlled examples of specific scan types. Nmap is a widely used network scanning tool that can make and transmit packets with custom TCP flag combinations, which makes it suitable for simulating attack behaviour.

Using Nmap, NULL and XMAS scan traffic were generated, a NULL scan, is where no TCP flags are set, was generated using the command:

```
sudo nmap -sN -Pn <target-ip>
```

An XMAS scan, which is where the FIN, PSH, and URG flags being set, was generated using:

```
sudo nmap -sX -Pn <target-ip>
```

Testing was carried out by replaying pre-processed packet datasets through the FPGA-based IDS. Packets were read from BRAM and transmitted sequentially into the processing pipeline, with the rate of packets being controlled using a delay between packet injections.

The system performs both per-packet classification and sliding window-based analysis. Individual packets are first classified based on protocol and flag information, after which traffic is aggregated within a fixed time window to identify higher-level behaviours such as flooding and volumetric attacks.

The following metrics were recorded during testing:

- Total number of packets processed
- Protocol and flag-based classification results
- Detection of attack types (e.g. SYN flood, ICMP flood, volumetric attack)
- Number of packets observed within each detection window

These metrics allow evaluation of both the correctness of packet classification and the behaviour of the sliding window detection mechanism under different traffic rates and threshold configurations.

5.2 Results

5.2.1 Threshold and Packet Rate Analysis

The intrusion detection system uses a time-based sliding window to detect high-volume traffic patterns. Two thresholds are defined: a volumetric threshold, which triggers when the total number of packets within a window exceeds a set value, and a flood threshold, which applies to protocol-specific traffic such as TCP SYN packets.

In the code, the thresholds were defined as:

```
uint16_t vol_threshold = 15;
uint16_t flood_threshold = 11;
```

Packet transmission rate is controlled by the delay between packet injections from BRAM, implemented using the `usleep()` function in the main program. This directly affects the number of packets that fall within each detection window. Three different rates were evaluated:

- `usleep(5000)` - increased packet rate
- `usleep(10000)` - base rate
- `usleep(20000)` - reduced packet rate

The test was performed using a dataset of normal network traffic taken from the CICIDS2019 dataset. This dataset provides realistic background traffic without significant attack patterns, making it suitable for analysing the effect of packet rate and threshold selection on false flags.

Table 1: Effect of packet transmission delay on volumetric detection

Delay	**Packets/window**	**Threshold**	**Detection result**
usleep(5000)	21, 43	Yes	Volumetric attack detected
usleep(10000)	12, 23, 22	Yes	Attack detected in Windows 2 and 3
usleep(20000)	5, 11, 12, 12, 12, 12	No	No volumetric attack detected

5.2.2 High Packet Rate: `usleep(5000)`

At a higher packet rate, a greater number of packets are processed within each 250 ms detection window, as shown in Table 1.

In this configuration, the packet count in every window exceeded the volumetric threshold, resulting in the detection of a volumetric attack. For example, window counts of 21 and 43 packets were observed, both well above the threshold of 15.

This shows that increasing the packet rate causes traffic to be concentrated within fewer windows, increasing the likelihood of threshold-based detection being triggered.

5.2.3 Baseline Packet Rate: `usleep(10000)`

At the baseline packet rate, packets are injected at a rate used in the other tests below, resulting in a more balanced distribution across detection windows when compared to the high and low packet rate configurations. The observed results are shown in Table 1.

In this configuration, the number of packets per window was observed to be 12, 23, and 22. The first window remained below the volumetric threshold of 20, while the second and third windows exceeded it. As a result, volumetric attack detection was triggered as it only needs to be in one window to trigger an attack.

This behaviour shows that the baseline rate lies close to the effective detection boundary. Unlike the high packet rate case, where all windows exceed the threshold, and the low packet rate case, where none do, the baseline rate produces mixed results. This demonstrates that small variations in packet timing can determine whether a threshold is crossed within a given window.

It is also worth noting that the total number of packets processed remains approximately constant across the different configurations, as the same dataset is replayed in each case. For the high and low packet rate tests, a total of 64 packets were observed, while the baseline configuration resulted in a slightly lower total of 57 packets within the analysed windows.

This discrepancy is attributed to the interaction between packet timing and the fixed window duration. At the baseline rate, some packets fall outside the measured window boundaries due to timing alignment, resulting in a lower observed total. This further illustrates the sensitivity of the system to packet timing, where even small changes in inter-packet delay can influence how traffic is distributed across detection windows.

5.2.4 Low Packet Rate: `usleep(20000)`

At a reduced packet rate, fewer packets are processed within each detection window, as shown in Table. 1.

In this case, packet counts per window ranged between 5 and 12, remaining below the volumetric threshold of 15. As a result, no volumetric attacks were detected, despite the same total number of packets being transmitted.

This highlights the sensitivity of the detection mechanism to packet timing. Although the overall traffic volume is unchanged, distributing packets over a longer period reduces the likelihood of exceeding the detection threshold within any given window.

5.2.5 Discussion

These results demonstrate that the detection system is strongly influenced by packet arrival rate relative to the window size. Higher packet rates concentrate traffic within a single window, increasing the likelihood of triggering threshold-based alerts, while lower rates distribute traffic across multiple windows, potentially avoiding detection.

This behaviour is expected for a sliding window-based detection mechanism and illustrates the importance of selecting appropriate threshold values and window sizes based on expected network traffic characteristics.

5.2.6 Syn Flood

Table 2: Comparison of SYN flood detection for different SYN dataset sizes

SYN Dataset Size	Observed packets per window	Detected attacks	Notes
64 packets	14, 23, 23, 4	SYN Flood; Volumetric Attack in Windows 2 and 3	Larger dataset; dynamic target tracking disabled
23 packets	14, 9	SYN Flood in Window 1 only	Reduced dataset; destination IP extracted as 192.168.50.4

The results obtained from the SYN packet dataset (Taken from CCSID 2019) show that the system is able to successfully identify SYN-based attack behaviour using both per-packet classification and sliding window analysis. Two configurations were evaluated, as shown in Table 2, a reduced dataset and a larger subset of the same dataset.

In the reduced configuration, a SYN flood was detected in the first window, where 14 packets were observed, exceeding the flood threshold of 11. Following windows contained fewer packets and did not trigger additional detections. In contrast, when a larger subset of the dataset was evaluated, multiple windows exceeded both the flood and volumetric thresholds. For example, packet counts of 23 per window resulted in simultaneous triggering of both SYN flood and volumetric attack alerts.

This behaviour is expected, as a higher concentration of SYN packets within a single detection window increases both the protocol-specific count and the total packet volume, allowing both thresholds to be exceeded. In compatison, when fewer packets are present within a window, only some windows may exceed the SYN flood threshold, and the volumetric threshold is less likely to be triggered.

Further analysis of the dataset, shown in Fig. 8, shows that the majority of SYN packets are directed towards a single destination IP address across multiple ports. This pattern is characteristic of SYN-based scanning or probing, where a host attempts to initiate connections across a range of ports. This behaviour is also reflected in the target tracking results, where only a single dynamic destination IP was identified. Although additional destination IPs are present within the dataset, they are associated with a small number of packets and do not trigger the detection window. As a result, only the main target exceeds the relevant thresholds and is recorded by the system. The observed traffic behaviour aligns with the IDS output, where repeated SYN packets within a short time interval result in the flood threshold being exceeded.

Overall, these results show that the detection system is sensitive to both traffic composition and the number of packets present within each detection window. The same underlying traffic can produce different detection outcomes depending on how packets are distributed across the fixed detection windows.

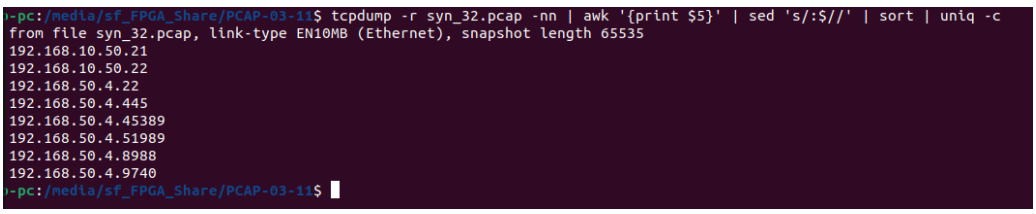


Figure 8: Extracted destination ports from the SYN packet dataset using `tcpdump`. The repeated destination IP with varying ports indicates SYN-based scanning or probing behaviour.

5.2.7 Mixed Packets

To further test the behaviour of the IDS under more realistic conditions, a mixed traffic scenario was made combining both attack and non-attack vectors. The generated scenario, shown in Fig. 9, is made up of of multiple hosts communicating through a central switch, with both normal traffic and attack traffic directed towards selected targets.

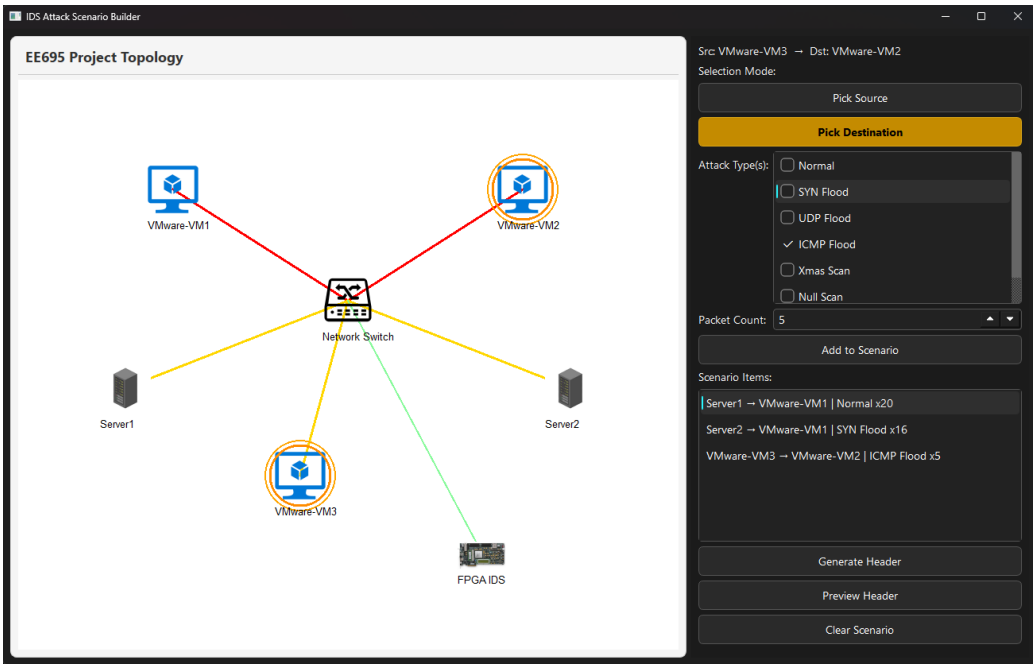


Figure 9: Mixed traffic scenario generated using the IDS attack scenario builder

In this test, three traffic sources and two destinations were defined. Server1 generated 20 normal packets directed towards VMware-VM1, representing a non attack vector. Then Server2 generated 16 TCP SYN packets towards the same destination, simulating a SYN flood attack. Additionally, VMware-VM3 generated a 5 ICMP packets directed towards VMware-VM2. It is worth noting that the source/destinatons were defined as:

Table 3: Network Node Configuration

Node Name	IP Address
Network Switch	`192.168.10.1`
VMware-VM1	`192.168.10.3`
VMware-VM2	`192.168.10.4`
Server1	`192.168.10.5`
Server2	`192.168.10.6`
VMware-VM3	`192.168.10.7`
FPGA IDS	`192.168.10.100`

This results in a total of 41 packets, with the majority of traffic concentrated on a single target. The results of this experiment are shown in Fig. 10, which is the serial monitor output coming from the FPGA. Packets were distributed across four detection windows, with counts of 5, 12, 12, and 12 packets respectively, which add up to the expected 41 packets. Three of these windows exceeded the volumetric threshold of 10 packets, and therefore a volumetric alert was triggered.

```
=====================================================
        EE695 FPGA Intrusion Detection
=====================================================


=====================================
RUN CONFIGURATION
=====================================
Window Size: 250 ms
Volumetric Threshold: 20
Flood Threshold: 15


=====================================
WINDOW SUMMARY
=====================================

Window 1:
Packets in window 1: 5
Attacks in window 1:

Window 2:
Packets in window 2: 12
Attacks in window 2:

Window 3:
Packets in window 3: 12
Attacks in window 3:

Window 4:
Packets in window 4: 12
Attacks in window 4:


=====================================
Attacks Detected
=====================================
Target: VMware-VM1 (192.168.10.3)
  [!] SYN Attack (16 SYN packets)
  [!] Volumetric Attack (36 total packets)

Dynamic targets seen: 0
=====================================
             DEMO DONE
=====================================
```

Figure 10: Window-level output for mixed traffic scenario

As we can see the IDS successfully identified a SYN flood targeting VMware-VM1. As shown in the final output, a total of 16 SYN packets were reported, exceeding the configured flood threshold and it also means the IDS system recorded all packets, as seen in Fig 9 there was 16 SYN packets generated.

In contrast, no alerts were generated for VMware-VM2. Although ICMP traffic was present, only 5 packets were observed, which remains well below the configured flood threshold. As a result, this traffic was correctly classified as non-malicious.

It is also noted that no dynamic targets were detected during this experiment. All observed destination IP addresses correspond to predefined hosts within the system, and therefore no additional target entries were created. This confirms that the dynamic target mechanism operates only on previously unseen destinations.

These results show that the IDS is capable of distinguishing between benign and malicious traffic within a mixed dataset, while also correctly associating detected attacks with specific targets. The system is able to detect protocol-specific attacks such as SYN floods even when packet arrivals are distributed across multiple detection windows, preventing individual windows from exceeding volumetric thresholds.

This behaviour shows the importance of combining per-packet classification with sliding window implementation. Sliding window analysis is effective for identifying short bursts of high traffic, whereas per-target statistics allow detection of attacks that are spread over time. This is important in mixed traffic scenarios, where malicious packets may be interwoven with normal traffic.

5.2.8 ICMP Flood

Table 4: ICMP flood detection results using CICIDS2017 dataset

Window	Packets in window	Detected attacks	Notes
Window 1	3	None	Below all thresholds
Window 2	23	ICMP Flood; Volumetric Attack	Exceeds both thresholds
Window 3	24	ICMP Flood; Volumetric Attack	Sustained high packet rate
Window 4	14	ICMP Flood	Exceeds flood threshold only

The ICMP flood detection capability of the system was tested using a dataset taken from the CICIDS2017 traffic capture. The results, as summarised in Table 4, show that the IDS is able to correctly identify ICMP-based attack behaviour.

In this experiment, packets were distributed across four detection windows containing 3, 23, 24, and 14 packets respectively. The first window remained below both the volumetric threshold of 15 and the flood threshold of 11, and therefore did not trigger any alerts. In contrast, the second and third windows exceeded both thresholds, resulting in simultaneous detection of ICMP flood and volumetric attacks.

The fourth window contained 14 packets, which is below the volumetric threshold but above the ICMP flood threshold. As a result, only the ICMP flood alert was triggered. This demonstrates that the system can distinguish between protocol-specific flooding behaviour and overall traffic volume, even when the two conditions do not occur simultaneously.

The final attack summary showed that all traffic was directed towards a single target, VMware-VM1 (192.168.10.3), with a total of 62 ICMP packets observed. This exceeds both the flood and volumetric thresholds when considered over the entire capture, resulting in both alerts being reported at the target level.

It should be noted that the IP address observed in this experiment originates from the real-world CICIDS2017 dataset. In this case, the destination address coincidentally matches one of the predefined IPs used within the test topology (192.168.10.3). As a result, the traffic is labelled as VMware-VM1 within the system, although it does not necessarily correspond to the same physical host.

These results confirm that the ICMP detection logic operates correctly on real-world traffic. Similar to the SYN flood case, detection is influenced by how packets are distributed across detection windows. High concentrations of packets within a single window result in both flood and volumetric alerts, whereas lower concentrations may only trigger protocol-specific detection.

Overall, this experiment shows that the IDS is capable of reliably detecting ICMP flood behaviour and correctly attributing it to the relevant target, even when traffic is distributed across multiple detection windows.

5.2.9 NULL Scan Detection

Table 5: NULL scan detection results using CICIDS2017 dataset

Window	Packets in window	Detected attacks	Notes
Window 1	1	None	Single packet
Window 2	24	NULL Scan; Volumetric Attack	Exceeds both thresholds
Window 3	23	NULL Scan; Volumetric Attack	Sustained high packet rate
Window 4	16	NULL Scan; Volumetric Attack	Above both thresholds

The NULL scan detection ability of the IDS was tested using a dataset derived from the CICIDS2017 traffic capture. A NULL scan is a TCP-based probing technique in which packets are transmitted with all control flags unset. Under normal TCP operation, at least one flag is expected, and therefore packets with no flags set are considered anomalous and indicative of scanning activity.

The results, summarised in Table 5, demonstrate that the system is able to correctly identify NULL scan behaviour using both per-packet classification and sliding window analysis.

In this experiment, packets were distributed across four detection windows containing 1, 24, 23, and 16 packets respectively. The first window contained only a single packet and therefore remained below the volumetric threshold of 15. However, a NULL scan alert was still triggered due to the presence of a packet with no TCP flags set, confirming that the per-packet classification logic is functioning correctly.

The remaining windows contained significantly higher packet counts, exceeding both the flood threshold of 11 and the volumetric threshold of 15. As a result, both NULL scan and volumetric attack alerts were triggered. This demonstrates that the system is capable of identifying both protocol-specific anomalies and high-volume traffic conditions within the same detection framework.

It is noted that no entries were produced in the final "Attacks Detected" section. This behaviour differs from previous experiments and is due to the software-based aggregation stage, which relies on consistent extraction of destination IP addresses. For this dataset, the destination information was not reliably captured during readback, preventing packets from being grouped into per-target statistics.

However, the window summary is generated directly by the FPGA and reflects real-time detection based on packet activity within each detection window. The correct identification of NULL scan behaviour across multiple windows therefore confirms that the hardware detection logic is operating as intended.

Overall, these results demonstrate that NULL scan detection is correctly implemented at the hardware level. While limitations in the software interface affect per-target reporting, the core detection mechanisms remain robust and capable of identifying anomalous TCP behaviour in real-world traffic.

5.2.10 XMAS Scan Detection

Table 6: XMAS scan detection results

Window	Packets in window	Detected attacks	Notes
Window 1	18	XMAS Scan; Volumetric Attack	Exceeds both thresholds
Window 2	23	XMAS Scan; Volumetric Attack	Sustained high packet rate
Window 3	23	XMAS Scan; Volumetric Attack	Consistent detection across windows

The XMAS scan detection capability of the IDS was evaluated using a dataset containing TCP packets with multiple control flags set. An XMAS scan is a type of probing technique in which packets are transmitted with several TCP flags enabled simultaneously (typically FIN, PSH, and URG), resulting in an unusual combination of flags that is not normally observed during standard TCP communication.

The results, summarised in Table 6, demonstrate that the system is able to consistently identify XMAS scan behaviour across all detection windows. In this experiment, packets were distributed across three windows containing 18, 23, and 23 packets respectively.

In all cases, the number of packets exceeded both the flood threshold of 11 and the volumetric threshold of 15. As a result, both XMAS scan and volumetric attack alerts were triggered in each window. This indicates that the IDS is able to reliably detect abnormal TCP flag combinations while also identifying high-volume traffic patterns.

No entries were produced in the "Attacks Detected" section for this experiment. As observed in the NULL scan case, this is attributed to the software-based aggregation stage, which depends on consistent extraction of destination IP addresses. For this dataset, destination information was not reliably captured, preventing packets from being grouped into per-target statistics.

Despite this limitation, the window-level results confirm that the FPGA-based detection logic is functioning correctly. The consistent identification of XMAS scan behaviour across all windows demonstrates that the system can accurately detect this class of anomaly in real-time.

Overall, these results show that the IDS is capable of reliably detecting XMAS scan activity using both per-packet classification and sliding window analysis, even when target-level aggregation is not available.

5.2.11 UDP Flood Detection

Table 7: UDP flood detection results

Window	Packets in window	Detected attacks	Notes
Window 1	18	UDP Flood; Volumetric Attack	Exceeds both thresholds
Window 2	23	UDP Flood; Volumetric Attack	Sustained high packet rate
Window 3	23	UDP Flood; Volumetric Attack	Consistent detection across windows

The UDP flood detection capability of the IDS was evaluated using a dataset containing a high volume of UDP packets directed towards a single destination. A UDP flood is a type of denial-of-service attack in which a target is overwhelmed with a large number of UDP packets, often causing resource exhaustion due to the need to process or respond to each packet.

The results, summarised in Table 7, demonstrate that the system is able to correctly identify UDP flood behaviour across all detection windows. In this experiment, packets were distributed across three windows containing 18, 23, and 23 packets respectively.

In all cases, the number of packets exceeded both the flood threshold of 11 and the volumetric threshold of 15. As a result, both UDP flood and volumetric attack alerts were triggered in each window. This indicates that the IDS is able to detect high-rate UDP traffic and correctly classify it as malicious based on packet volume and protocol type.

No entries were produced in the "Attacks Detected" section for this experiment. As observed in previous cases, this is attributed to the software-based aggregation stage, which relies on consistent extraction of destination IP addresses. For this dataset, the destination information was not reliably captured, preventing packets from being grouped into per-target statistics.

Despite this limitation, the window-level results confirm that the FPGA-based detection logic is functioning correctly. The consistent identification of UDP flood behaviour across all windows demonstrates that the system can accurately detect high-volume UDP-based attacks in real time.

Overall, these results show that the IDS is capable of reliably detecting UDP flood activity using both sliding window analysis and per-packet classification, further validating the effectiveness of the system across multiple protocol types.

5.3 System Limitations

While the proposed FPGA-based IDS demonstrates effective detection of multiple attack types, several limitations were identified during implementation and testing.

One key limitation is the reliance on on-chip BRAM for packet storage and processing. As the available BRAM on the FPGA is limited, only a relatively small number of packets can be buffered at any given time. This constrains the size of datasets that can be replayed and necessitates the use of reduced or compact packet representations. As observed during testing, larger datasets can lead to instability or incomplete processing, particularly when additional features such as dynamic target tracking are enabled.

Dynamic target tracking introduces further resource constraints. The allocation of memory for storing per-target statistics consumes valuable BRAM resources, which limits the number of concurrent targets that can be tracked. In this implementation, the number of dynamic targets is therefore restricted, and in some experiments this feature was disabled to ensure reliable system operation. This highlights a trade-off between detection capability and hardware resource usage.

Another limitation arises from the software-based aggregation stage. The final "Attacks Detected" summary depends on consistent and reliable extraction of destination IP addresses from the hardware. As observed in the NULL and XMAS scan experiments, unstable or inconsistent metadata readback can prevent packets from being correctly grouped, resulting in missing per-target summaries despite correct detection at the window level. This indicates that the communication interface between hardware and software is a critical component of the overall system reliability.

From a detection perspective, the IDS is primarily threshold-based and focuses on a predefined set of attack signatures, including SYN floods, ICMP floods, UDP floods, and TCP flag-based scans. While effective for these cases, this approach may not generalise well to more complex or subtle attack patterns. For example, attacks such as ACK floods or low-rate distributed attacks may not be detected if they do not exceed the configured

thresholds within a given window. Similarly, the system does not perform deep packet inspection or stateful flow analysis, limiting its ability to detect application-layer or evasion-based attacks.

Finally, the use of fixed detection windows introduces sensitivity to packet timing. As demonstrated in earlier experiments, the same traffic can produce different detection outcomes depending on how packets are distributed across windows. This can lead to borderline cases where attacks are inconsistently detected, particularly when traffic rates are close to threshold values.

Overall, these limitations highlight the challenges of implementing an IDS on resource-constrained hardware. While the system achieves efficient real-time detection for a range of common attack types, further improvements in memory management, metadata handling, and detection algorithms would be required to support more complex and scalable network monitoring scenarios.

5.4 Comparison with Software-Based IDS

Table 8: Comparison between FPGA-based IDS and software IDS

Feature	**Software IDS (e.g. Snort)**	**FPGA IDS (This Work)**
Latency	High (ms)	Low (hardware-level)
Power Consumption	High	Low
Parallel Processing	Limited	Highly parallel
Flexibility	High	Moderate

6 Ethical Considerations

7 Conclusions and Future Work

Department of Electronic Engineering, Maynooth University

4th YEAR PROJECT RISK ASSESSMENT FORM Rev. 3

NAME AND STUDENT NUMBER:	PROJECT NAME: FPGA-based Intrusion Detection for DoS Attack
SUPERVISOR:	PROJECT LOCATION: **E.107**

BRIEF DESCRIPTION OF PROJECT

Using
Vc707 board to detect and prevent DOS/DdOS attacks

Hazards, Risk [High(H) Medium (M) Low (L)], and Control Measures

HAZARD	Risk	Controls
Data Privacy Breach	**L**	**Use anonymized or synthetic datasets for training and testing. Ensure no personally identifiable information (PII) is captured or stored. Restrict access to project data.**
System Misconfiguration	**L**	**Develop a detailed implementation plan and follow strict version control. Use a separate, isolated test network for all experiments to prevent impact on the main network.**
Hardware Damage	**L**	**Follow Electrostatic Discharge (ESD) safety procedures (e.g., use of wrist straps). Ensure correct voltage and power supply is used. Operate in a temperature-controlled environment.**
Data Loss or Corruption	**L**	**Regularly back up all project code, datasets, and configuration files to a separate, secure location (e.g., cloud storage, external drive). Use version control systems like Git.**
Eye Strain from prolonged screen use	**L**	**Ensure proper room lighting and adjust monitor brightness. Take regular breaks.**
Electrical Shock when handling the VC707 board and peripherals	**L**	**Always disconnect the power supply before connecting or disconnecting cables or components. Handle the board by its edges**
Mental Fatigue and Burnout	**L**	**Adhere to a structured work schedule with defined start and end times. Ensure you get adequate sleep and schedule time for hobbies and social activities completely unrelated to the project.**

Identified risks should be discussed with your supervisor and a safe system of work agreed. A more in-depth risk assessment may be required after initial review. Do not proceed until form is signed off.

WEEE Compliance & Recycling: Indicate below how you intend to manage and dispose of your project's material during and post project work.

N/A, project has no material that needs disposing of

Further Controls Required